

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Thaís dos Reis Soares
Matrícula: 825322

Detecção de Bad Smells e Refatoração Segura

Disciplina: Teste de Software

1. Análise de *Smells*

1.1 *Divergent Change*

O primeiro *smell* encontrado foi o *Divergent Change*. Esse problema ocorre quando uma classe precisa ser modificada por diferentes motivos e em diferentes contextos. Ao realizar a análise manual da classe *ReportGenerator*, foi possível perceber que ela possuía diferentes responsabilidades. Ela aplicava as regras de negócio relacionadas aos usuários, geração de diferentes tipos de arquivo, formatação visual deles, entre outras.

Isso é um problema, porque faz com que a classe esteja sujeita a muitas mudanças, e qualquer alteração em uma funcionalidade específica pode impactar as outras, o que dificulta a manutenção. Além disso, se surgir novas funcionalidades, como um novo tipo de usuário ou de arquivo, será necessário incluir na classe essa nova característica, o que a tornará ainda maior e, conseqüentemente, prejudicará sua legibilidade e compreensão, o que dificulta a realização de testes. E, com o tempo, ela poderá evoluir para uma *God Class*, um outro tipo de *smell* de código.

1.2 *Long Method*

Na classe *ReportGenerator*, havia um só método que realizava tudo, concentrando muitas funcionalidades em um único bloco de código. Ele era responsável por gerar o relatório, filtrar itens, aplicar regras de negócio, entre outras responsabilidades. Portanto, ele se caracterizava como um *Long Method*.

Isso é um problema, pois um método grande e com muitas responsabilidades distintas é difícil de entender, o que dificulta sua manutenção e sua testabilidade. Qualquer alteração em um trecho desse método poderia inserir erros em diversas funcionalidades.

1.3 *Duplicated Code*

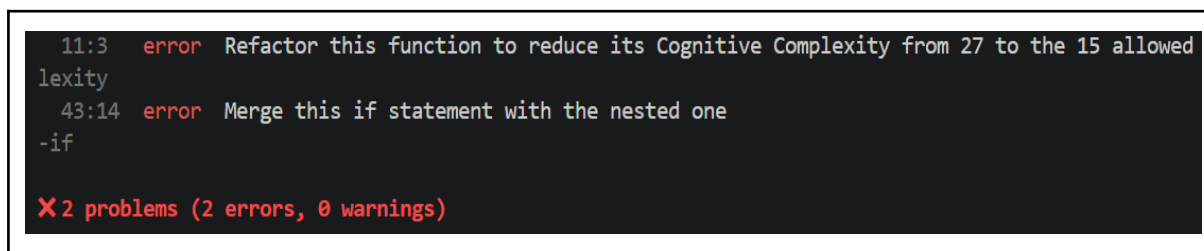
O código original possuía vários trechos praticamente idênticos repetidos em diferentes partes do método. Isso acontecia na geração de linhas CSV e HTML, no cálculo do total, nas estruturas condicionais para “admin” e “user”, entre outros casos.

Isso é um problema, porque ao se realizar uma alteração, ela precisa ser feita em várias partes do código diferentes, e isso aumenta a chance de surgir inconsistências e erros. Esse problema também afeta os testes, porque

comportamentos iguais acabam espalhados pelo sistema. Assim, um erro corrigido em um trecho pode permanecer em outro trecho duplicado, causando comportamentos inconsistentes.

2. Relatório da Ferramenta

A ferramenta ESLint foi utilizada para analisar o código fonte e encontrar *bad smells*. A seguir, a figura 1 mostra o resultado da primeira execução dela, antes da refatoração.



```
11:3 error Refactor this function to reduce its Cognitive Complexity from 27 to the 15 allowed
lexity
43:14 error Merge this if statement with the nested one
-if

X 2 problems (2 errors, 0 warnings)
```

Figura 1 - resultado do ESLint antes da refatoração

Conforme ilustrado, a ferramenta identificou problemas no arquivo. Isso foi importante para confirmar os *smells* encontrados na análise manual e identificar detalhes técnicos que poderiam passar despercebidos.

Um dos principais pontos analisados foi a complexidade cognitiva, que mede o quanto é difícil entender o funcionamento de um método. A ferramenta mostrou que o método *generateReport* ultrapassou o limite aceitável configurado, indicando que havia muitas condicionais e regras misturadas no mesmo trecho.

Com isso, a análise deixou de ser apenas uma opinião e passou a ter dados concretos, ajudando a mostrar claramente a necessidade da refatoração e, por isso, serviu como guia para refatorar o código.

3. Processo de Refatoração

O *smell* mais crítico que corrigi foi o *duplicated code*, principalmente relacionado às diversas estruturas condicionais espalhadas que verificavam o tipo de arquivo e, assim, geravam mais código repetido. A seguir, serão apresentados alguns trechos dessas estruturas repetidas.

```
// --- Seção do Cabeçalho ---
if (reportType === 'CSV') {
  report += 'ID,NOME,VALOR,USUARIO\n';
} else if (reportType === 'HTML') {
  report += '<html><body>\n';
  report += '<h1>Relatório</h1>\n';
  report += '<h2>Usuário: ${user.name}</h2>\n';
  report += '<table>\n';
  report += '<tr><th>ID</th><th>Nome</th><th>Valor</th></tr>\n';
}
```

Figura 2

```
if (reportType === 'CSV') {
  report += `${item.id},${item.name},${item.value},${user.name}\n`;
  total += item.value;
} else if (reportType === 'HTML') {
  const style = item.priority ? 'style="font-weight:bold;"' : '';
  report += `<tr ${style}><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
  total += item.value;
}
```

Figura 3

```
// Users comuns só veem itens de valor baixo
if (item.value <= 500) {
  if (reportType === 'CSV') {
    report += `${item.id},${item.name},${item.value},${user.name}\n`;
    total += item.value;
  } else if (reportType === 'HTML') {
    report += `<tr><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
    total += item.value;
  }
}
```

Figura 4

Para corrigir esse *smell*, utilizei a técnica *Replace Conditional with Polymorphism* para substituir os vários *if/else* por classes responsáveis pelo comportamento de cada tipo de relatório. Em vez de verificar constantemente se o relatório era CSV ou HTML, o sistema passou a delegar essa responsabilidade para objetos específicos.

Também utilizei a técnica *Extract Class* para separar a lógica de formatação em novas classes, *CsvFormatter* e *HtmlFormatter*. Dessa forma, cada classe passou a ter apenas a responsabilidade de montar seu próprio formato de relatório.

A seguir, a figura 5 ilustra as classes criadas.

```
class CsvFormatter {
  buildHeader() {
    return 'ID,NOME,VALOR,USUARIO\n';
  }

  buildRow(item, user) {
    return `${item.id},${item.name},${item.value},${user.name}\n`;
  }

  buildFooter(total) {
    return `\nTotal,,\n${total},,\n`;
  }
}

class HtmlFormatter {
  buildHeader(user) {
    let header = '<html><body>\n';
    header += '<h1>Relatório</h1>\n';
    header += '<h2>Usuário: ${user.name}</h2>\n';
    header += '<table>\n';
    header += '<tr><th>ID</th><th>Nome</th><th>Valor</th></tr>\n';
    return header;
  }
}
```

Figura 5

O método *createFormatter* recebe o tipo de arquivo e retorna um objeto daquela instância, como mostra a figura 6.

```
function createFormatter(reportType) {
  if (reportType === 'CSV') return new CsvFormatter();
  if (reportType === 'HTML') return new HtmlFormatter();
  throw new Error(`Tipo de relatório desconhecido: ${reportType}`);
}
```

Figura 6

Com isso, basta chamar os métodos do objeto criado, sem a necessidade de estruturas condicionais, como é apresentado pela figura 7.

```
generateReport(reportType, user, items) {  
  const formatter = createFormatter(reportType);  
  
  const visibleItems = this.filterItemsByRole(user, items);  
  const { rows, total } = this.buildRows(formatter, user, visibleItems);  
  
  const header = formatter.buildHeader(user);  
  const footer = formatter.buildFooter(total);  
  
  return (header + rows + footer).trim();  
}
```

Figura 7

4. Conclusão

Para concluir, é importante ressaltar a importância de se utilizar testes como uma rede de segurança durante o processo de refatoração. Um código pode estar funcionando corretamente, mas ter sua estrutura repleta de *bad smells*, como foi visto neste trabalho. Neste caso, os testes passam, mas isso não significa que o código esteja escrito da melhor forma. Com isso, se torna necessário realizar uma refatoração, porém ela não pode introduzir erros no sistema, e isso pode ser garantido com a execução dos testes.

Por fim, vale destacar que um código escrito de forma inadequada prejudica diversos aspectos do desenvolvimento de *software*, como a compreensão, a manutenção e a realização de testes. Por isso, a redução de *bad smells* é extremamente importante, pois melhora a legibilidade do código, facilita sua manutenção e torna os testes mais simples e confiáveis.